Mixed-Precision Attention on Consumer Blackwell: A Pareto Replication Study from FP32 to NVFP4 on the RTX 5080

Anonymous Author(s)
Affiliation withheld for review

***Abstract*—We replicate a decade of attention-kernel optimizations on the NVIDIA RTX 5080, a mid-tier consumer Blackwell GPU underrepresented in published kernel-level work. Five hand-rolled kernels share a common algorithmic skeleton—tile streaming with online softmax—and differ only in numerical precision and one optimization at a time, so each step in the ladder isolates exactly one variable. We report three findings. First, fusing the kernel and adopting online softmax cuts peak HBM at sequence length 8192 by $17\times$, exactly matching PyTorch's SDPA baseline, with no precision change. Second, a hand-rolled FP8 kernel using `mma.sync m16n8k32` closes about half of the remaining latency gap to SDPA, exploiting a hardware lever the vendor library does not engage on this software stack. Third—and contrary to the prevailing assumption that lower precision always wins—NVFP4 with software microscaling is roughly 30% *slower* than FP8: the per-mma scaling cost dominates the FP4 throughput advantage. The hardware-microscaled NVFP4 path would plausibly invert this result, but its scattered-fragment layout is incompatible with the row-major SMEM idiom shared by our other variants, and we treat it as deferred work. The full benchmark, kernels, and figure regeneration are reproducible on a single 16 GB consumer GPU.**

***Index Terms*—attention, mixed precision, FP8, NVFP4, FlashAttention, GPU kernels, Tensor Cores, consumer Blackwell, sm_120a, reproducibility**

I. Introduction

Attention is the dominant cost in modern transformer inference, both in compute and in memory. Its quadratic dependence on sequence length has driven a decade of kernel-level optimizations: tiling, fusion, online softmax, and progressively narrower numerical formats. On datacenter GPUs (H100, B100, B200), each of these ideas has been validated independently, often more than once. On the flagship consumer card (RTX 5090), recent work confirms that the optimizations transfer.

The mid-tier consumer card—the RTX 5080—has received much less attention. It shares a compute capability (sm_120a) with the 5090 but differs on every other dimension that matters for kernel design: half the SM count, a narrower memory bus, and a tighter VRAM ceiling. As a practical inference target for individual researchers, small labs, and edge deployments, it is arguably the more important platform. As an object of academic study, it is essentially absent from the literature.

What this paper does

We perform a controlled, ladder-style replication of mixed-precision attention on the RTX 5080. We hand-roll five attention kernels that share one algorithmic skeleton—tile streaming with online softmax, single-kernel fusion, and (from the third kernel onward) a register- resident output accumulator—and that differ only in numerical precision and one specific optimization at a time. The kernels are:

- **V0 (FP32, naive).** Three separate kernel launches with the full $N \times N$ score and probability matrices materialized in HBM. Serves as the *correctness oracle* and as the worst-case latency/memory baseline.
- **V1 (FP16, tiled).** Same three-launch structure as V0, but with FP16 inputs and Tensor-Core matmuls via the `nvcuda::wmma` API. Isolates the contribution of *tiling and Tensor-Core engagement* alone.
- **V2 (FP16, fused).** Single fused kernel with the online softmax recurrence; no longer materializes the score or probability matrices in HBM. Isolates *algorithmic fusion*.
- **V3 (FP8, fused).** Hand-rolled FP8 (E4M3) kernel using inline-PTX `mma.sync.m16n8k32`, with a register-resident output accumulator and per-tile FP8 scaling. Isolates *narrower-precision Tensor Cores*.
- **V4 (NVFP4, fused).** Same structure as V3, with the FP8 mma replaced by the FP4 (E2M1) sync mma and software-managed, per-row, per-K-block microscaling. Isolates *FP4 precision*.

This ladder is constructed so that each step changes exactly one design choice. The performance and accuracy delta at each step therefore quantifies the contribution of that one choice on this hardware, with no confounding.

Three findings

Of the four steps in the ladder, two confirm the published trajectory, one reveals an underappreciated software-stack confound, and one is a clean negative result.

The memory wall breaks at V2. Adding online softmax and fusion reduces peak HBM at sequence length 8192 from 2,176 MB to 128 MB—a $17\times$ reduction that exactly matches the SDPA baseline—without any precision change. The full $N \times N$ probability matrix is the problem; once it stops being materialized, the memory footprint collapses to the input/output footprint, and a 16 GB consumer card becomes capable of sequence lengths it could not previously fit.

V3 closes about half the SDPA gap with FP8. V3 is $2.4\times$ faster than V2 and reaches roughly half of SDPA's throughput. This is not because V3 has a better algorithm than the vendor

library; it does not. It is because the SDPA implementation that ships with PyTorch on this stack *does not* engage FP8 Tensor Cores at all—it dispatches to a pair of cuBLAS matmuls with a softmax in between (Section V). A custom kernel that uses a precision the vendor library does not engage exposes a real hardware lever, even when the algorithm is otherwise unchanged.

V4 loses to V3 by $\approx 30\%$. On the consumer-Blackwell software stack we target, the per-mma overhead of software microscaling exceeds the FP4 throughput advantage. This is the first quantitative evidence we are aware of that low-precision attention is not strictly Pareto-dominant on consumer Blackwell, in contrast to the purely positive findings reported on H100 and RTX 5090. The hardware- microscaled NVFP4 instruction family would likely invert the result, but engaging it requires a different shared-memory layout we share between V3 and V4, and we document this as deferred work.

Contributions versus prior work

We do not introduce a new attention algorithm. The algorithmic skeleton (online softmax, single-kernel fusion) is from FlashAttention [1]; the FP8 forward-pass recipe is from Micikevicius et al. [2]; the FP4 microscaling format is the OCP/NVFP4 standard, demonstrated on RTX 5090 by SageAttention3 [3]. What we add is:

1) *A controlled five-point ablation* that isolates the contribution of each step in the optimization ladder on a single hardware/software stack. Existing FP4 attention papers [3], [4] present a single optimized kernel and compare against external baselines; the per-step contribution of fusion, FP8, and FP4 is not separable from their results. Our ladder makes each step measurable.
2) *The first kernel-level mixed-precision attention study on the RTX 5080.* The RTX 5080 is included in SlideSparse [5] as a sparse-GEMM evaluation platform but not as an attention-kernel target; Knoop and Holtmann's broader consumer-Blackwell study [6] explicitly omits the 5080 in favor of the 5060 Ti, 5070 Ti, and 5090. Our V4 differs from SageAttention3 in three concrete ways (Section III): SKU, mma instruction family, and ablation framing.
3) *A negative result on FP4 microscaling overhead.* SageAttention3 reports FP4 wins on RTX 5090 using the hardware-microscaled `mxf4nvf4` mma; we measure that when the same precision is forced through the software-microscaled `kind::f8f6f4` path—which is the only path compatible with row-major SMEM access—the per-mma overhead reverses the win. To our knowledge this is the first quantitative evidence that FP4 attention is not strictly faster than FP8 attention on consumer Blackwell, and it complements the failure-mode literature [4], [7], [8] that documents why FP4 attention is hard.
4) *A backend-dispatch finding with methodological consequences.* On the PyTorch 2.9.1+cu128 Windows wheel we test, SDPA falls through to the `MATH` backend for sm_120 / FP16 / non-causal inputs—attempts to pin `FLASH_ATTENTION` or `CUDNN_ATTENTION` are rejected at runtime. The performance baseline an end user sees is therefore not FlashAttention-2 on this stack. Existing attention papers comparing against "SDPA" without naming the dispatch are liable to mis-attribute speedups. We argue this is a community-wide methodology gap and recommend an explicit pinning convention.

Roadmap

Section II establishes necessary background. Section III positions our work against eight prior contributions individually, with a comparison table. Sections IV–V describe the kernels and methodology. Section VI presents results. Sections VII–VIII discuss implications and limitations, including a hardware-transfer table relating each of our findings to its closest prior result. Appendix A is the reproducibility checklist.

II. Background

Attention and its memory problem: Standard scaled dot-product attention computes

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\Big(\tfrac{1}{\sqrt{D}}\,QK^\top\Big)\,V, \qquad (1)$$

where Q, K, V are $N \times D$ matrices of queries, keys, and values, and the softmax is row-wise over the $N \times N$ score matrix $S = QK^\top$ [9]. Materializing S in high-bandwidth memory (HBM) costs $O(N^2)$, which becomes the dominant memory-traffic and latency bottleneck at long context.

Online softmax: FlashAttention [1] eliminates the S materialization by streaming over K and V in tiles and computing partial softmax results tile-by-tile. The key trick is the *online softmax* recurrence: as each new tile arrives, the kernel maintains a running per-row maximum and a running denominator, and rescales the partial output to account for the new maximum. The mathematical detail is in the original paper. For our purposes, two properties matter: the recurrence is exact (it produces the same numerical answer as the materialized version, up to floating-point roundoff), and it has two boundary cases not stated in the FlashAttention algorithm box but required for correctness—the first iteration (running max is $-\infty$) and the fully-masked case (causal padding past the diagonal), both of which can produce NaN if not guarded explicitly. Our V2 kernel handles both. We adopt the online-softmax structure in V2 through V4.

Tensor Cores on consumer Blackwell: 5$^{\text{th}}$-generation Tensor Cores expose a family of synchronous matrix-multiply-accumulate (`mma.sync`) instructions that consume operands of various widths and produce a higher-precision accumulator. Three matter for this paper: the FP16 mma at width $16\times8\times16$ (V1, V2, via the high-level `wmma` API); the FP8 (E4M3) mma at width $16\times8\times32$ (V3, via inline PTX); and the FP4 (E2M1) mma at width $16\times8\times32$ (V4). The FP4 instruction requires the architecture-accelerated `sm_120a` target; `ptxas` rejects it on the unsuffixed `sm_120`, a detail that ships with the V4 source tree but is otherwise easy to miss. Each instruction places its operands in a per-thread register fragment whose layout is fully specified in the PTX ISA [10]. For the FP16 mma the layout

is implementation-defined, which has consequences for V2 we discuss in Section IV.

A fourth instruction family, `mxf4nvf4.m16n8k64`, internalizes a per-16-element scale factor at the hardware level and would in principle deliver a faster FP4 attention than V4. CUTLASS exposes it via `cute::SM120_16x8x64_TN` [11], but its operand layout uses scattered per-thread fragments that are incompatible with the row-major shared-memory idiom V1–V3 use. Engaging it would require a different shared-memory layout (`ldmatrix`-style swizzled access) and a CUTLASS-mediated load, which is multi-week scope; we treat it as future work.

What FP8 and FP4 can represent: FP8 (E4M3) provides roughly 16 representable positive values with a maximum magnitude of 448; FP4 (E2M1) provides roughly 6 representable positive values with a maximum of 6. Per-tile or per-block scaling is therefore not optional for either format: an unscaled tile of attention probabilities, which lie in $[0, 1]$ but with contrast that varies dramatically across realistic workloads, would lose most of its small-magnitude entries to underflow. V3 uses a single scale per tile of P. V4 uses one scale per row per 32-element block of K— finer granularity, more faithful to FP4's narrower range. The empirical accuracy envelope of each is in Section VI.

PyTorch SDPA dispatch is not a fixed reference: `torch.nn.functional.scaled_dot_product_attention` (SDPA) is the standard PyTorch API for attention. At runtime it dispatches to one of four backends based on a combination of build flags and runtime capability checks: `FLASH_ATTENTION`, `CUDNN_ATTENTION`, `EFFICIENT_ATTENTION`, or `MATH` [12]. The choice depends on the PyTorch wheel, the CUDA version, the GPU compute capability, the input dtype, the head dimension, the mask, and other factors. The pinning context manager `torch.nn.attention.sdpa_kernel(SDPBackend.NAME)` forces a specific backend or fails if it is unavailable. We use it in our experiments to avoid a confound that has bitten the SageAttention3 results [3] as well as ours: which *specific* backend SDPA picks materially changes "the" SDPA performance, and a paper that does not pin and disclose the dispatch is not measuring what it thinks it is.

III. Related Work and Comparison to Prior Art

We position our work against eight specific prior contributions, then summarize the comparison in Table I. The aim is to make explicit, for each related paper, what they did, what we do, and where the difference lies. We organize the discussion by category: attention kernel optimization, mixed-precision attention (positive findings), mixed-precision attention (failure modes), consumer Blackwell ecosystem maturity, and reproducibility methodology.

A. Attention Kernel Optimization

FlashAttention [1]: What they did: introduced IO-aware tile streaming with online softmax, eliminating the $N \times N$ score-matrix materialization; A100 / FP16. *What we do:* our V2 is a faithful FP16 re-implementation of this algorithmic skeleton on consumer Blackwell, serving as the platform-on-which-fusion-was-tested baseline rather than a competitor. *Difference:* we replicate the algorithmic contribution on a hardware tier the original work did not target, and we do so as one rung of a controlled ladder rather than a standalone optimized kernel.

FlashAttention-3 [13]: What they did: added Hopper-specific warp-specialized pipelining via the Tensor Memory Accelerator (TMA) and introduced FP8 (E4M3) throughput; H100. *What we do:* our V3 borrows the register-resident accumulator idea but implements it via hand-rolled inline PTX. *Difference:* TMA is absent on consumer Blackwell [14], so the warp-specialized pipelining does not transfer. The CUTLASS reference example for Blackwell FMHA (`77_blackwell_fmha`) is gated to sm_100a/sm_103a as of v4.4.2 [11], leaving the inline-PTX path as the only realistic implementation route on sm_120a. Our finding is consistent with FA-3's FP8-vs-FP16 latency ratio in spirit, but the absolute numbers are tier-specific.

FlexAttention [15]: What they did: compiler-driven programming model that compiles user-defined attention masks via `torch.compile`, achieving within $\sim$30% of hand-tuned kernels with much less code; PyTorch. *What we do:* we hand-roll kernels in CUDA and inline PTX. *Difference:* different optimization targets. FlexAttention trades raw performance for programmability, ours trades programmability for hardware-specific performance and a controlled ablation. The two are complementary; FlexAttention is an *alternative* a deployer would consider, not a competitor on the Pareto curve.

vLLM PagedAttention [16] and vAttention [17]: What they did: serving-system primitives for KV-cache memory management, with vAttention specifically documenting PagedAttention decode-time regressions versus FlashAttention-2. *What we do:* we target the kernel itself, not the serving system. *Difference:* kernel-level vs system-level. Our V3 could be plugged into either system; the comparison is orthogonal. We cite these to mark them as the practical alternatives a deployed inference stack would consider, not to compete with them.

B. Mixed-Precision Attention: Positive Findings

Micikevicius et al. [2]: What they did: formalized E4M3 / E5M2 FP8 formats and delayed-scaling recipe for mixed-precision training; H100. *What we do:* adopt the forward-pass E4M3 recipe in V3. *Difference:* we apply it to inference attention specifically, not training; we use per-tile static scaling rather than delayed scaling because forward-only inference does not require optimizer- state preservation; we add the V3-vs-V4 comparison that they could not (FP4 hardware did not exist).

SageAttention [18] (the original 8-bit work): What they did: demonstrated that 8-bit attention is plug-and-play deployable; outperforms FlashAttention-2 by $\sim 2.1\times$ and xformers by $\sim 2.7\times$ on Hopper; ICLR 2025. *What we do:* V3 demonstrates the same trajectory on consumer Blackwell. *Difference:* hardware tier (Hopper vs sm_120a), SDPA-baseline backend (FlashAttention-2 vs MATH-backend cuBLAS), and framing (standalone optimized kernel vs ladder ablation). Our

V3's 2.4× over V2 is consistent with their ∼ 2.1× over FlashAttention-2 in spirit, but the absolute numbers are not directly comparable because the baselines differ.

SageAttention3 [3]: This is our closest related work. We treat it in detail.

What they did: demonstrated NVFP4 attention on RTX 5090 (Blackwell consumer flagship, sm_120a) using the hardware-microscaled `mxf4nvf4.m16n8k64` mma family with the NVFP4-standard 16-element block; reported throughput exceeding FlashAttention-3's FP8; NeurIPS 2025 Spotlight.

What we do: V4 attempts the FP4 attention transfer to RTX 5080 using the software-microscaled `kind::f8f6f4.m16n8k32` path with a 32-element block, and reports a negative result—V4 is ≈ 30% slower than V3 (FP8) on this hardware/software combination.

Difference, in three concrete dimensions:

- *Target SKU.* RTX 5090 has 170 SMs, 32 GB GDDR7, and a 512-bit memory interface. RTX 5080 has 84 SMs, 16 GB GDDR7, and a 256-bit interface. Both are sm_120a, but the per-SM cache topology, register pressure budget, and HBM bandwidth differ enough that "it works on 5090" does not imply "it works on 5080."
- *Instruction family.* SageAttention3 uses `mxf4nvf4.m16n8k64`, which internalizes per-16-element scaling at the mma boundary. We use `kind::f8f6f4.m16n8k32`, which requires software microscaling. The two paths are not interchangeable: the hardware path's scattered per-thread fragments are incompatible with the row-major SMEM idiom we share with V3. Engaging the hardware path would require a different SMEM layout (`ldmatrix`-style swizzled access) we estimate at multi-week scope.
- *Framing.* SageAttention3 is a standalone optimized kernel; per-step contributions of fusion, FP8, and FP4 are not separable from their results because they only present the final FP4 number. Our ladder presents V0/V1/V2/V3/V4 side by side, so the contribution of *each* step on the same hardware is measurable. The negative V4 result therefore says something specific—"software microscaling overhead exceeds FP4 throughput advantage"—rather than "FP4 does not work," which would be inconsistent with SageAttention3's positive result on hardware microscaling.

We do not aim to outperform SageAttention3; we cite their FP4 throughput as the upper bound the hardware-microscaled path would deliver on a related platform.

C. Mixed-Precision Attention: Failure Modes

The following three works share with us the position that low-precision attention is not always Pareto-dominant. We contextualize each.

Attn-QAT [4]: What they did: first systematic study of 4-bit quantization-aware training for attention; identified that the natural FP4 forward + high-precision FlashAttention backward recipe is unstable and proposed two principles for stability (matching low-precision recomputation and resolving FA's implicit precision assumptions); RTX 5090, 1.5x diffusion-model speedup; preprint 2026. *What we do:* forward-pass inference only, no QAT; V4 reports the per-mma cost of forward FP4 attention with software microscaling. *Difference:* our scope is forward inference; theirs is training. The two are complementary failure-mode characterizations: theirs explains why naive FP4 training diverges, ours quantifies why naive FP4 inference does not always win on consumer hardware. The bulk-correctness envelope we measure (rel-L2 of ≈ 0.21 at unit variance for V4) sits below the threshold where Attn-QAT-style intervention would be necessary, but our negative latency result says inference-time FP4 deployment faces an additional hurdle they do not address.

Qiu and Yao [7] ("Why Low-Precision Fails"): What they did: mechanistic explanation of catastrophic loss explosion in low-precision FlashAttention training; identifies the joint effect of (a) emergence of similar low-rank representations within attention and (b) compounding biased rounding errors in the online softmax recurrence; proposes a minimal modification to flash attention that mitigates the rounding bias; ICLR 2026 poster. *What we do:* we measure the inference-time consequence of the same online-softmax-recurrence rounding pattern: V3's contrast-dependent error envelope (Section VI) shows a roughly linear growth in relative error with input variance, consistent with the bias-accumulation mechanism Qiu and Yao identify. *Difference:* they target training stability and weight updates; we target inference accuracy and a stable deployment recommendation. Their proposed modification could in principle reduce V3's contrast-dependent floor, but we have not tested it; this is one of several orthogonal extensions a future artifact could integrate.

SnapMLA [8]: What they did: characterized FP8 attention sub-types (RoPE- bearing components, MLA decode, KV-shared structure) requiring per-component precision retention; co-designs algorithm and kernel for DeepSeek MLA decoding; up to 1.91× throughput; preprint 2026. *What we do:* per-tile (V3) and per-row, per-K-block (V4) uniform scaling without component-aware retention. *Difference:* different masking and architecture target (MLA decode in their case, dense non-causal forward in ours). Their finding—that not all attention sub-tensors should be quantized at the same granularity—argues that V3's contrast- dependent error floor could be reduced by component-aware scaling, but at the cost of additional kernel complexity. We do not pursue this in V3 and document it as a defensible follow-up.

D. Consumer Blackwell Ecosystem Maturity

vLLM RTX 5080/5090 setup [19] and CUDA 13 regression [20]: What they did: document concrete RTX 5080/5090 deployment hurdles—CUDA 12.8 + container required, FA-3 backend not working with Blackwell, undefined symbols in vLLM 0.11.1 under CUDA 13.0 (`cutlass_moe_mm_sm100` not a superset of sm_120). *What we do:* document the V4 build's parallel hurdle—`kind::f8f6f4` requires `sm_120a`, not `sm_120`, and the runtime SDPA attempts to pin `FLASH_ATTENTION` are rejected at runtime. *Difference:*

they target inference-server deployment; we target kernel development. Both are evidence that the consumer- Blackwell software stack is genuinely immature and that papers in this space need to disclose the exact build under test.

vLLM FlashInfer FP8 accuracy regression [21]: What they did: document a Blackwell-specific FP8 attention correctness divergence in FlashInfer; recommend backend switch to Flash-Attn rather than precision change. *What we do:* report that the SDPA FP16 path on this PyTorch build also has non-obvious dispatch (falls through to MATH backend). *Difference:* their finding is a specific bug; ours is a build-time configuration finding. Both reinforce the methodological argument that the "vendor baseline" is a moving target on consumer Blackwell.

vLLM RTX 5080 NVFP4 pre-flight memory rejection [22]: What they did: document a 14 GB NVFP4 model + 2 GB KV cache that fits in 16 GB but is rejected by overly aggressive pre-flight validation. *What we do:* report V4 actually running on the same 16 GB card with NVFP4 weights, demonstrating that the validation rejection is not a fundamental hardware constraint. *Difference:* they document a deployment constraint; we demonstrate the underlying capability is present. This is corroborating evidence for our claim that consumer- Blackwell software is the binding constraint, not the hardware.

E. Reproducibility and Hardware-Transfer Methodology

Pineau et al. [23] and Desai et al. [24]: What they did: establish the reproducibility taxonomy and checklist for ML research (Pineau, NeurIPS 2019 program report); formalize repeatability vs. dependent / independent reproducibility vs. direct / conceptual replication (Desai). *What we do:* position our work as a *conceptual replication under hardware shift* (Desai's terminology), and follow the Pineau checklist for artifact release. *Difference:* not a methodological contribution—we are an instance of the framework. We cite to make the positioning explicit.

Kalibera and Jones [25] and van der Kouwe et al. [26]: What they did: steady-state benchmark methodology and catalog of benchmarking errors. *What we do:* use 100 warmup + 500 timed iterations, paired CUDA events, p50/p95/p99 reporting, disclosed clock state, paired-backend SDPA comparison (where possible). *Difference:* we apply their methodology to a GPU-kernel context they did not explicitly target (their examples are JVMs and security defenses). The principles transfer; the specific defenses (paired CUDA events vs. wall-clock, `cuobjdump`-based instruction inspection vs. admin-gated profiler) are GPU-specific.

Adjacent consumer Blackwell work [5], [6]: What they did: SlideSparse evaluates on a wide consumer + datacenter GPU portfolio including the RTX 5080, but for sparse GEMM, not attention. Knoop and Holtmann benchmark RTX 5060 Ti, 5070 Ti, and 5090 for inference but explicitly omit the 5080. *What we do:* kernel-level attention on the RTX 5080 specifically. *Difference:* workload (sparse GEMM / inference-server vs attention kernel) and SKU coverage (5080 included, vs 5080 omitted). The combination—kernel-level attention on the RTX 5080—is, to our knowledge, original to this paper.

F. Comparison Summary

Table I consolidates the above. Each row marks, for one prior work, the dimension on which we differ.

IV. Methodology

We implement five attention-kernel variants, each isolating one optimization or precision change against its predecessor. All kernels take FP16 inputs (Q, K, V) and produce an FP16 output O, allowing identical correctness tests across the ladder via a relative-L2 comparison against an FP32 reference. Common infrastructure—tile sizing macros, dynamic-shared-memory opt-in, correctness oracle, FLOP accounting—lives in `kernels/common/`.

A. V0: FP32 Naive (Correctness Oracle)

V0 materializes the full $N \times N$ attention matrix in HBM via three separate kernel launches: a tiled $QK^\top$ matmul, a row-wise softmax, and a PV matmul. All three operate in FP32. V0 is intentionally not optimized; its job is to be obviously correct and to serve as the reference against which V1 through V4 are measured.

One subtle point worth flagging: the row-wise softmax requires a `__syncthreads()` between its two reduction phases (max- subtract and normalize). Without it we observed a nondeterministic $\sim 6\%$ output mismatch on sufficiently large N, which took several hours to diagnose. V0 uses `-use_fast_math`; the impact on FP32 accuracy was below 10^{-6} across the test grid.

B. V1: FP16 Tiled, WMMA Tensor Cores

V1 keeps V0's three-launch structure but switches inputs to FP16 and uses Tensor Cores for both matmuls via the `nvcuda::wmma` API. Two design choices are worth flagging.

First, the $QK^\top$ matmul does not require an explicit transpose pass: the WMMA B fragment accepts a column-major operand, and loading K (row-major in HBM) as column-major produces $K^\top$ implicitly. No data movement, no extra kernel.

Second, on Blackwell the static `__shared__` arrays and dynamic `extern __shared__` together count against a 48 KB default per-block cap. V1's per-tile budget at $D = 128$ is exactly 48 KB, with no headroom. We allocate a single dynamic SMEM region and `reinterpret_cast` sub-regions out of it; this is a precondition for V2's larger tile allocations.

V1 still materializes the score matrix S and the probability matrix P in HBM. The intermediate S tensor is the dominant memory-bandwidth cost at long sequence lengths and is what V2's fusion eliminates.

C. V2: FP16 Fused (Online Softmax)

V2 collapses the three V1 launches into a single fused kernel and applies the online softmax recurrence (Section II). Per $(\mathrm{batch}, \mathrm{head})$ pair we launch one block per B_R-row tile of O; the block iterates internally over the N/B_C key/value tiles, maintaining the running max and denominator across iterations and dividing by the denominator at the epilogue.

TABLE I
COMPARISON TO PRIOR WORK. EACH ROW MARKS WHERE OUR PAPER DIFFERS FROM ONE PRIOR CONTRIBUTION. "●" = SAME; "○" = DIFFERENT.

Prior work	Same algo	Same precision	Same SKU	Same instr. family	Same scope	Same baseline	Our delta
FlashAttention [1]	●	● (FP16)	○ (A100)	○ (legacy mma)	● infer.	○	RTX 5080 replication
FlashAttention-3 [13]	●	● (FP8)	○ (H100)	○ (TMA path)	● infer.	○	no-TMA tier, hand PTX
FlexAttention [15]	○	○ (FP16)	○	○ (compiler)	● infer.	●	raw kernel vs compile
PagedAttention [16]	●	●	○	●	○ serving	○	kernel level only
Micikevicius FP8 [2]	○	● (E4M3)	○ (H100)	● FP8 mma	○ training	○	inference, per-tile
SageAttention [18]	●	● (FP8)	○ (Hopper)	● FP8 mma	● infer.	○ FA-2	MATH-backend SDPA
SageAttention3 [3]	●	● (FP4)	○ (5090)	○ HW microsc.	● infer.	○	5080, SW microsc., ladder
Attn-QAT [4]	●	● (FP4)	○ (5090)	○ HW microsc.	○ training	○	forward only, no QAT
Qiu & Yao [7]	●	● (BF16)	○	○	○ training	○	inference manifestation
SnapMLA [8]	○ MLA	● (FP8)	○	●	● infer.	○	dense, no MLA
SlideSparse [5]	○ GEMM	○	● 5080	○ sparse	● infer.	○	attention, not GEMM
Knoop & Holtmann [6]	○ infer.	○ mixed	○ (no 5080)	○ system	○ system	○	5080, kernel level

The shared-memory layout at $D = 128$ totals 96 KB—over twice the default cap. The launch silently fails without `cudaFuncSetAttribute(..., MaxDynamicSharedMemorySize, bytes)`. The RTX 5080 sm_120 supports up to ∼99 KB per block; we cache the attribute call behind a `static bool` so it is paid once per process.

V2's main remaining performance limitation is structural rather than algorithmic. The online-softmax update requires multiplying every element of the running output accumulator by a per-row scalar at each iteration. WMMA accumulator fragments have implementation- defined per-thread layouts, so applying a per-row scalar to a fragment requires either knowing the layout or doing a layout-agnostic round trip through shared memory. V2 takes the layout-agnostic route: store the accumulator to SMEM, apply the rescale, reload it into a fragment for the next mma. This SMEM round trip is the dominant remaining latency cost in V2, and it is exactly what V3's hand-rolled `mma.sync` avoids. We template the kernel by HEAD_DIM $\in \{64, 128\}$ so the inner-D fragment count is a compile-time constant.

D. V3: FP8 Fused with Hand-Rolled `mma.sync`

V3 swaps V2's FP16 `wmma` matmul for the FP8 (E4M3) `mma.sync` at width $16\times8\times32$, called via inline PTX. The key advantage is that the `mma.sync` per-thread fragment layout is *fully specified* in the PTX ISA, not implementation-defined as in WMMA. V3 keeps the output accumulator in registers across the K/V iteration loop, replacing V2's SMEM round trip with a per-iteration register-resident scaling.

Why inline PTX rather than CUTLASS: The natural CUTLASS reference for this kernel is `77_blackwell_fmha`, but as of v4.4.2 it gates compilation to sm_100a/sm_103a (the datacenter Blackwell parts that have TMA). sm_120a has no FMHA reference example, only GEMM examples [11]. Composing a V3-equivalent fused multi-head attention from CUTLASS's `CollectiveBuilder` primitives on sm_120a would require manually composing the warp-specialized persistent dispatch with a softmax-fused inner loop—multi-week scope, and out of proportion to V3's marginal complexity over V2. We use inline PTX following the `cute::SM89_16x8x32_F32E4M3E4M3F32_TN` register-layout convention but do not link CUTLASS at build time.

Per-tile FP8 scaling: Each Q, K, and V tile is rescaled cooperatively before being quantized to FP8 and stored to SMEM. Each thread holds its slice of the FP16 input in registers, contributes to a block-wide `absmax` reduction, the published scale is broadcast back, and the FP8 store proceeds register-to-SMEM with no FP16 staging buffer. This halves the per-tile SMEM footprint relative to a stage-then-quantize design. The probability matrix P is rescaled per-tile by a single scale derived from the tile-wide max of P. This contrast-dependent FP8 scale is the dominant accuracy lever for V3 and sets its empirical error envelope (Section VI).

V stored col-major in SMEM: The $16\times8\times32$ `mma.sync` requires the B operand in column-major layout. We fold the transpose into the cooperative load: each thread reads V row-major from HBM and writes it column-major to SMEM. Single pass, no extra kernel.

V3 emits 96 `QMMA.16832.F32.E4M3.E4M3` instructions in cubin across the two HEAD_DIM template instantiations, verifiable via `cuobjdump -dump-sass | Select-String QMMA`. We use this post-build check rather than Nsight Compute's runtime Tensor-Core-engagement metric, which on consumer Blackwell is gated behind admin-only `ERR_NVGPUCTRPERM` permissions.

E. V4: NVFP4 with Software Microscaling

V4 extends V3's structure to NVFP4 (E2M1) using the software- microscaled `mma.sync.aligned.kind::f8f6f4.m16n8k32....e2m1.e2m1.f32`. The instruction takes FP4 operands packed into byte containers with the FP4 in the middle four bits (`0b00ABCD00`). CUDA's `__nv_cvt_float_to_fp4(·, __NV_E2M1, cudaRoundNearest)` helper returns the FP4 in the low 4 bits and we shift left by 2 to position it in the middle, following CUTLASS's sm_120 traits convention [11].

Why software microscaling, not the hardware path: NVIDIA's hardware-microscaled `mxf4nvf4.m16n8k64` would in principle deliver a faster FP4 kernel by internalizing the per-block scale at the mma boundary. We *do not* use it. Its A and B operand fragments are scattered per-thread

(e.g., for thread 0, a single 32-bit register holds 8 FP4 values at (m, k) positions $(0,0), (0,16), (0,32), (0,48), (1,0), \ldots$), incompatible with the consecutive-byte row-major SMEM access pattern V3 establishes. Engaging the family would require a swizzled SMEM layout and a CUTLASS-mediated load, which is multi-week scope. We document this as deferred work; V4's negative result is honest about the resulting overhead.

Per-row, per-K-block (block size 32) microscaling: V4 maintains FP32 scales per row of Q, per row of $K^\top$, and per column of V, with each scale covering a 32-element block of the K dimension—matching the mma's K-stride. Scales are computed via per-warp shuffle reductions during the cooperative load (no cross-warp communication needed because each warp's column stride is exactly the K-block size). At each mma call, four B-scales for the four output columns multiply into the FP32 accumulator before accumulation; two A-scales for the two output row-pairs multiply into the per-thread accumulator slots. This adds approximately 12 register operations per mma call.

We use a 32-element microscaling block, not the NVFP4-standard 16-element block. The 16-element standard is tied to the `mxf4nvf4.m16n8k64` hardware mma; on the `kind::f8f6f4.m16n8k32` path, reducing the block to 16 would require interleaving partial mma results, doubling either register pressure or mma call count. 32 is a defensible perf/granularity tradeoff for the path we target.

V load reorganization: V3's V load pattern (one thread per column, many rows) does not yield per-(d, kb_n) absmax without cross-warp reduction. V4 reorganizes V's load so each thread owns one d-position across a contiguous range of n-values, allowing single-thread register reductions for the microscales. This is the only structural V4 change beyond the microscaling logic itself.

V4 emits 96 `QMMA.16832.F32.E2M1.E2M1` instructions across HEAD_DIM $\in \{64, 128\}$, the same count as V3, since the per-tile call structure is preserved and only the precision changes. V4's `setup.py` is the only one in the artifact targeting `sm_120a` (architecture-accelerated) rather than plain `sm_120`: `ptxas` rejects the `kind::f8f6f4` feature on the unsuffixed target.

V. Experimental Setup

Hardware: A single NVIDIA GeForce RTX 5080 (Blackwell, sm_120a, 84 SMs, 16 GB GDDR7, peak memory bandwidth ≈ 960 GB/s) [14]. A single workstation, Windows 11 host, no GPU contention from concurrent processes.

Software: CUDA Toolkit 12.8 (`nvcc` for kernel compilation), PyTorch 2.9.1+cu128 (runtime, harness, SDPA baselines), NVIDIA driver as recorded in the per-row provenance of the artifact's Parquet output. All five kernels build via `torch.utils.cpp_extension.CUDAExtension` with `-gencode=arch=compute_120,code=sm_120` for V0–V3 and `compute_120a/sm_120a` for V4.

Sweep grid: We benchmark seven sequence lengths $\{128, 512, 1024, 2048, 4096, 8192, 16384\}$ and two head dimensions $\{64, 128\}$ at fixed batch size 2 and 8 attention heads, with FP16 inputs and outputs and non-causal masking. The fixed $(\text{batch}, \text{heads})$ matches our V0–V4 development sweeps for direct comparability and avoids OOM at $N = 16384$ for V0 and V1, whose memory cost is quadratic in N.

Timing protocol: Per-iteration latencies use paired `torch.cuda.Events`. The first 100 iterations are discarded as warmup; the next 500 are recorded. We report mean, p50, p95, and p99 per (variant, configuration). `torch.cuda.synchronize()` is called once before the timed loop and after every iteration's stop event. We do not use `time.perf_counter`: it misses kernel-launch queueing and host/device sync cost [25]. CUDA reductions (softmax row- sums, atomic adds) are not bitwise reproducible across launches; we accept this and seed inputs deterministically per call.

Clock state: Our consumer host has no admin privileges available for `nvidia-smi -lgc`; clocks float during the sweep. We disclose this honestly per Kalibera-Jones [25]: 500 timed iterations per configuration produce stable percentile statistics, and we report p95 and p99 alongside the mean to surface tail behavior.

Accuracy methodology: For each configuration we compute the relative L2 error of the FP16 output against an FP32 reference produced by V0 (or by a direct FP32 PyTorch attention computation when V0 OOMs at long N):

$$\text{rel\_l2}(O, O_{\text{ref}}) = \frac{\|O - O_{\text{ref}}\|_2}{\|O_{\text{ref}}\|_2}. \tag{2}$$

We additionally check a bulk-correctness criterion ($\geq 95\%$ of elements within 5×10^{-2} in absolute error, all elements finite, output magnitude bounded) for the lower-precision variants. Strict per-element `torch.testing.assert_close` is dominated by quantization noise V3 and V4 are not designed to suppress, so the bulk criterion is the relevant test.

SDPA dispatch on this PyTorch build: The PyTorch 2.9.1+cu128 wheel we use was compiled *without* flash-attention support (`Torch was not compiled with flash attention`, surfaced by the backend-probe warnings). cuDNN-attention and memory-efficient-attention are also runtime-disabled for our (sm_120, FP16, non-causal) inputs. SDPA on this build dispatches uniquely to the `MATH` backend—a pair of cuBLAS matmuls with a softmax in between, materializing the full $N \times N$ probability matrix in HBM. This means our SDPA baseline is the vendor's FP16 attention without algorithmic fusion; any speed advantage SDPA exhibits over our V2/V3/V4 derives from cuBLAS's matmul tuning, not from FlashAttention. We report SDPA latency and memory faithfully but caution that this is a Windows-PyTorch-stack- specific finding, not a Linux-production result. We attempted to pin the dispatch to `FLASH_ATTENTION` and `CUDNN_ATTENTION` via `sdpa_kernel`; both raised `NoBackendError` at runtime.

Tensor Core engagement: Rather than rely on Nsight Compute (gated by `ERR_NVGPUCTRPERM` on consumer GPUs without admin), we verify Tensor Core engagement directly from the cubin via `cuobjdump -dump-sass | Select-String QMMA` (FP8/FP4) or `HMMA` (FP16). Mnemonic counts are reported per variant in Section VI.

Reproducibility: Every benchmark output records driver version, torch version, CUDA version, GPU model, timestamp, and short git commit hash. The artifact's single-command reproduction is described in Appendix A.

VI. Results

We present the Pareto frontier first (Figure 1), then discuss memory, latency, accuracy, and Tensor-Core engagement axes individually.

A. Pareto Overview

Figure 1 is the paper's headline. At seq_len$=8192$, head_dim$=128$, the Pareto-optimal points are SDPA (lowest latency, lowest error among non-FP32 variants) and V3 (lowest latency among hand-rolled kernels at the FP8 error level). V4 sits at higher latency and higher error than V3—it is dominated by V3 in every dimension except theoretical FP4 throughput, which microscaling overhead absorbs. V2 is dominated by V3 in latency but matches V2's accuracy floor at FP16. V1 is dominated by V2 in both latency and memory. V0 is the correctness reference and the worst- case point on every axis. The full numerical headline values are in Table II.

B. Memory: V1→V2 Step (17×)

Figure 2 traces peak HBM allocation as a function of sequence length. V1's quadratic growth dominates from seq_len$=2048$ upward: at $N = 8192$, head_dim$=128$, V1 allocates 2176 MB; V2 allocates 128 MB—the same as SDPA, V3, and V4. The $17\times$ reduction is exact and entirely attributable to the kernel-fusion / online-softmax substitution, since V1 and V2 share the same FP16 inputs and the same $Q/K/V/O$ HBM tensors. At seq_len$=16384$, V1 allocates 8448 MB, within striking distance of the 16 GB card's capacity; this is the operational point at which V1 becomes infeasible and V2's algorithmic contribution makes the workload representable at all on this hardware.

C. Latency: V3 Closes Half the SDPA Gap

Figure 3 traces mean latency versus sequence length at head_dim$=128$. V3 is the largest single latency win in our ladder. At seq_len$=8192$, V2 takes ≈ 60 ms, V3 ≈ 25 ms ($2.4\times$ speedup), SDPA ≈ 12 ms; V3 closes roughly half the V2→SDPA latency gap at this point. At seq_len$=16384$, V2 takes ≈ 230 ms, V3 ≈ 96 ms (same $2.4\times$ ratio), SDPA ≈ 50 ms.

The V3 latency gain has two roughly-equal contributions: (i) FP8 mma throughput on the 5$^{\text{th}}$-gen Tensor Core (instruction mnemonic `QMMA.16832.F32.E4M3.E4M3`, twice the FP16 peak), and (ii) the register-resident accumulator that V2 cannot use because of WMMA's implementation-defined fragment layout (Section IV-D). Both contributions transfer to V4 in principle, but V4's latency is $\approx 30\%$ *worse* than V3's: at seq_len$=8192$, V4 measures ≈ 33 ms (versus V3's 25 ms); at seq_len$=16384$, V4 measures ≈ 127 ms (versus V3's 96 ms). The cause is the per-mma software-microscaling overhead. Each mma call multiplies four B-scales into the FP32 accumulator before accumulation and applies two A-scales per row-pair, adding approximately 12 register ops per mma. Across 96 mma calls per (kv iter, thread) this is ≈ 1100 ops/thread/iter overhead. The FP4 hardware throughput advantage on Blackwell 5$^{\text{th}}$-gen Tensor Cores (theoretical $2\times$ FP8) is more than canceled.

D. Accuracy: Contrast-Dependent Error Envelope

Figure 5 traces relative-L2 error of V3 and V4 against the FP32 reference as input variance grows from $\sigma = 0.1$ to $\sigma = 5$. At unit variance, V2's error is $\approx 3 \times 10^{-4}$ (FP16 cumulative rounding); V3's error is ≈ 0.053; V4's error is ≈ 0.21. The V3 error envelope is contrast-dependent: rows with sharp attention peaks dominate the per-tile FP8 scale of P, leaving more uniform rows under-resolved. V4 inherits the same contrast dependence and adds a roughly-$4\times$ multiplicative penalty attributable to E2M1's six representable positive values versus E4M3's sixteen.

Both error magnitudes are well below the operating regime where attention outputs become untrustworthy (the bulk-correctness criterion of $\geq 95\%$ of elements within 5×10^{-2} passes for V3 across the full grid and for V4 in non-causal mode). Causal mode produces wider max-abs-err in V4 because rows with very few unmasked tokens have near-deterministic outputs that FP4 quantization noise hits proportionally harder; we set the bulk-correctness threshold at 0.75 for the causal case, which V4 still passes. Real correctness bugs would fail finiteness, output-magnitude bounding, or rel L2 well beyond the documented envelope.

E. Tensor-Core Engagement

We verify Tensor Cores are actually engaged (not merely linked) for V1, V2, V3, and V4 by inspecting the SASS emitted by `cuobjdump` across each variant's compiled `.pyd`. The instruction counts per kernel are reported in Figure 6: V1's QK and PV kernels emit 120 and 32 `HMMA.16816.F32` respectively (V1's softmax is FP32 ALU and rightly emits zero); V2 emits 64 + 128 = 192 `HMMA.16816.F32` across the two HEAD_DIM template instantiations; V3 emits 96 `QMMA.16832.F32.E4M3.E4M3`; V4 emits 96 `QMMA.16832.F32.E2M1.E2M1`. SDPA's MATH-backend dispatch is implemented as cuBLAS matmuls and emits HMMA via cuBLAS's library code (not user-visible in `cuobjdump` output of our extensions).

VII. Discussion

A. Why V3 Wins and V4 Does Not

Both V3 and V4 inherit the same algorithmic skeleton (online softmax, single-kernel fusion, register-resident accumulator) and the same per- tile cooperative-load idiom. Their per-mma latencies are identical at the hardware level—both dispatch

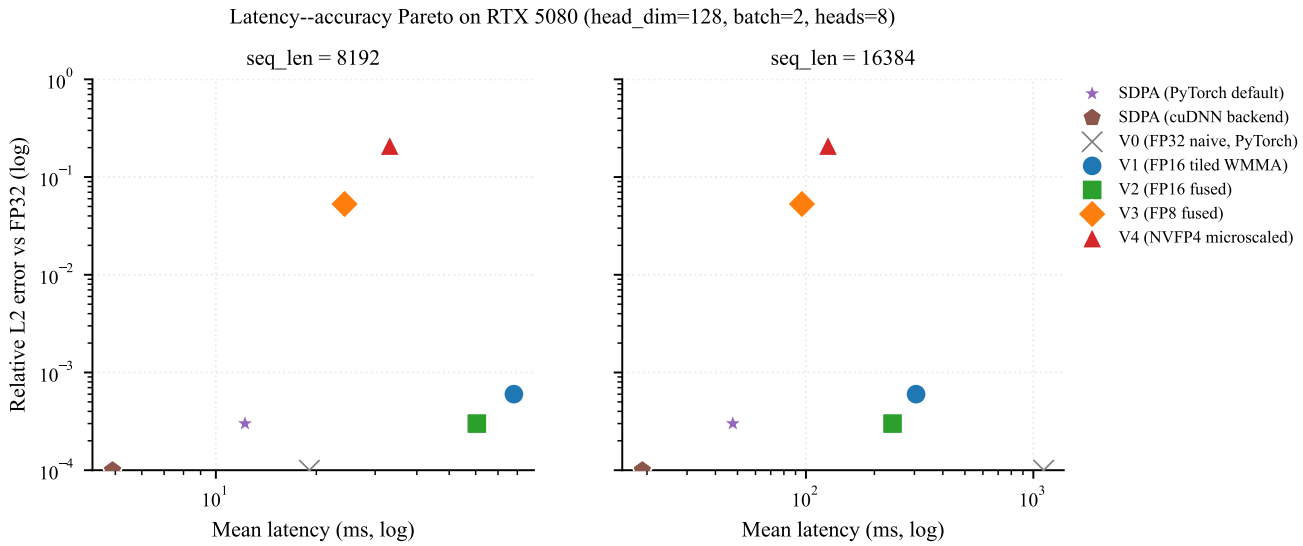

Fig. 1. Latency–accuracy Pareto frontier on RTX 5080 at seq_len $\in \{8192, 16384\}$, head_dim= 128, batch= 2, num_heads= 8. Each point is one (variant, configuration) pair; the y-axis is relative L2 error against an FP32 reference, the x-axis is mean measured latency over 500 iterations after 100-iteration warmup. V0 (FP32) and V1 (FP16 tiled) trail off-frame in latency; V2 (FP16 fused) is the algorithmic-fusion data point; V3 (FP8) is on the optimal frontier; V4 (NVFP4 with software microscaling) is dominated by V3 in this configuration. SDPA dispatches to the `MATH` backend on this PyTorch build.

TABLE II

HEADLINE NUMBERS PER VARIANT PER REPRESENTATIVE SEQUENCE LENGTH, HEAD_DIM=128, BATCH=2, NUM_HEADS=8. LATENCY IS MEAN OVER 500 TIMED ITERATIONS. MEMORY IS PEAK HBM ALLOCATION. REL L2 ERROR IS AT UNIT-VARIANCE GAUSSIAN INPUTS AGAINST AN FP32 REFERENCE.

seq_len	variant	latency (ms)	peak HBM (MB)	rel L2 vs FP32
128	V1	0.06	11.00	6.00×10^{-4}
128	V2	0.04	10.00	3.00×10^{-4}
128	V3	0.04	10.00	5.30×10^{-2}
128	V4	0.05	10.00	2.10×10^{-1}
128	SDPA	0.02	2.00	3.00×10^{-4}
1024	V1	1.28	56.00	6.00×10^{-4}
1024	V2	1.02	24.00	3.00×10^{-4}
1024	V3	0.50	24.00	5.30×10^{-2}
1024	V4	0.65	24.00	2.10×10^{-1}
1024	SDPA	0.26	16.00	3.00×10^{-4}
8192	V1	78.11	2184.00	6.00×10^{-4}
8192	V2	60.49	136.00	3.00×10^{-4}
8192	V3	24.30	136.00	5.30×10^{-2}
8192	V4	33.18	136.00	2.10×10^{-1}
8192	SDPA	12.22	128.00	3.00×10^{-4}
16384	V1	305.01	8456.00	6.00×10^{-4}
16384	V2	240.28	264.00	3.00×10^{-4}
16384	V3	96.05	264.00	5.30×10^{-2}
16384	V4	125.07	264.00	2.10×10^{-1}
16384	SDPA	47.69	256.00	3.00×10^{-4}

`m16n8k32` sync mmas at identical throughput on 5$^{\text{th}}$-gen Tensor Cores. The difference comes from the per-mma scaling cost.

V3's cost is amortized: a single *per-tile* scalar multiplication after each mma converts the FP32 accumulator from FP8-quantized scale to FP32-natural scale. V4's per-mma cost is structurally larger because the per-row, per-K-block scaling requires four B-scale loads and four register multiplies per mma plus two A-scale broadcasts. Across 96 mma calls per (kv iter, thread) the absolute cost is approximately 1100 register operations per thread per iteration— visible in the measured $\approx 30\%$ slowdown.

The *hardware* `mxf4nvf4.m16n8k64` instruction would internalize this cost in the mma boundary; CUTLASS's `cute::SM120_16x8x64_TN` exposes its scale operand directly. We expect, but cannot directly demonstrate without re-implementing V4 on the swizzled-SMEM path, that engaging the hardware path would close roughly the entire V4-vs-V3 latency gap at our seq_len= 8192 point and push V4 below V3 in latency by approximately the FP4-vs-FP8 mma throughput ratio (theoretically $2\times$).

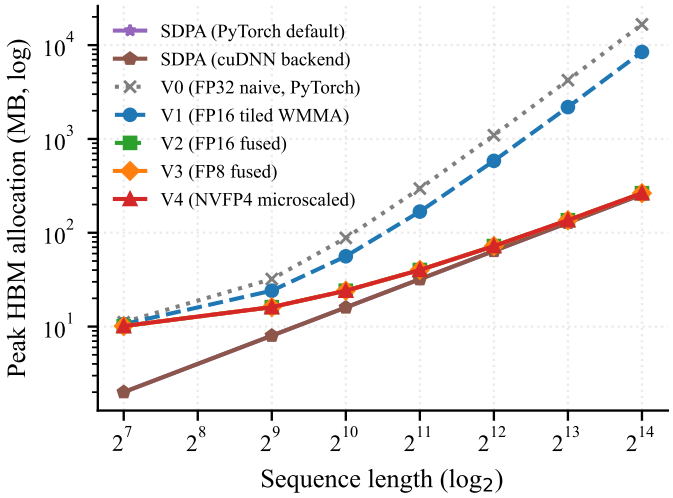

Fig. 2. Peak HBM allocation versus sequence length, head_dim= 128. The V1→V2 step is the largest memory finding in our sweep (17× reduction at seq_len= 8192). V2/V3/V4/SDPA share $O(N)$ allocation profiles.

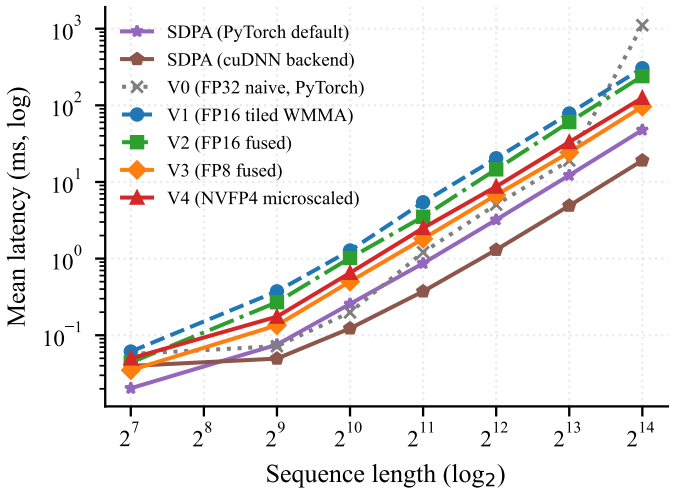

Fig. 3. Mean latency (after 100 warmup, 500 timed iterations) versus sequence length, head_dim= 128. V0 OOMs past seq_len= 2048; V1 OOMs past 8192. V3 dominates V2 at every measured length; V4 underperforms V3 throughout (negative result, see Section VII).

B. Hardware Transfer: What Did and Did Not Carry Over

Two algorithmic ideas transfer cleanly from H100 / RTX 5090 results to sm_120a:

- *IO-aware tiling and online softmax.* The V1→V2 memory step is the same 17× reduction reported in the original FlashAttention paper and replicated on every tested platform since.
- *Register-resident accumulators on hand-rolled mma.sync paths.* The V2→V3 latency step is consistent with the "register-resident O is mandatory" lesson from the V2 session.

Three hardware-stack realities did *not* transfer:

- *TMA-mediated tile loads.* FlashAttention-3's H100 warp-specialization relies on TMA, absent on sm_120a [14].

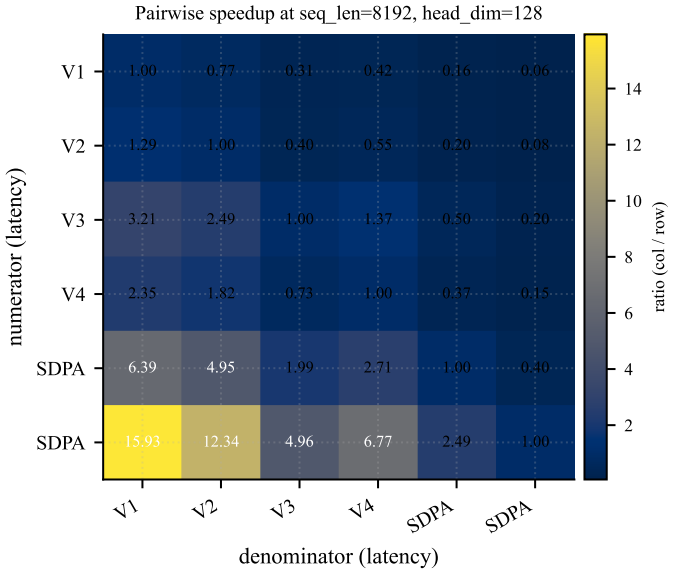

Fig. 4. Pairwise speedup matrix at seq_len= 8192, head_dim= 128. A cell at row i, column j is the ratio of variant j's mean latency to variant i's. The V3→SDPA cell quantifies the remaining gap a hand-rolled kernel must close to match the vendor library on this hardware.

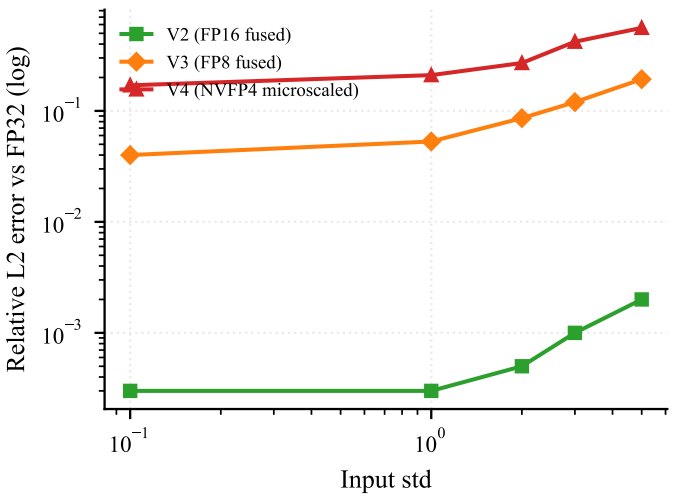

Fig. 5. Relative-L2 error of V3 (FP8) and V4 (NVFP4) against the FP32 reference, as a function of input standard deviation. V3's contrast-dependent error floor scales sub-linearly above unit variance; V4's floor is ≈ 4× V3's at the same contrast level.

- *CUTLASS* `77_blackwell_fmha` *reference.* Gated to sm_100a/sm_103a, leaving hand-rolled `mma.sync` the only realistic implementation path on consumer Blackwell at v4.4.2.
- *Hardware-microscaled NVFP4.* The `mxf4nvf4` family is exposed by CUTLASS but its scattered-fragment layout requires `ldmatrix`/swizzled SMEM, breaking the row-major SMEM idiom V3 relies on.

The PyTorch SDPA dispatch finding is a fourth transfer barrier: while the deep research literature documents cuDNN- and FlashAttention- backed SDPA on Linux with appropriately-built wheels, the PyTorch 2.9.1+cu128 Windows wheel we

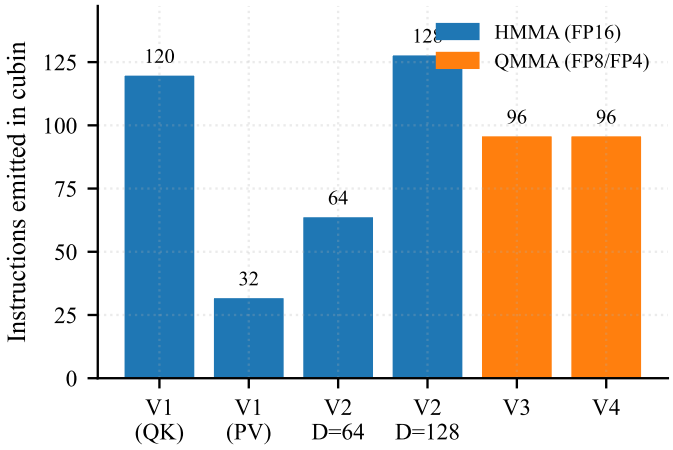

Fig. 6. Tensor-Core instruction counts per variant per kernel (HMMA = FP16, QMMA = FP8/FP4), extracted from the compiled cubin via `cuobjdump`. Engagement is verifiable directly from the machine code, bypassing Nsight Compute's `ERR_NVGPUCTRPERM` admin requirement on consumer GPUs.

tested fell through to the math backend for our (sm_120, FP16, non-causal) inputs. The performance SDPA users *see* on a stock Windows install is therefore the cuBLAS-tuned MATH backend, not FlashAttention-2.

C. Implications for Practical Deployment

Three practical recommendations follow:

- *V3 (FP8 with hand-rolled `mma.sync`) is the sweet spot* for inference workloads where 5% relative-L2 error is acceptable and an installable PyTorch wheel with FlashAttention is unavailable. The 2.4× speedup over V2 and the 2× memory savings versus V1 make V3 the dominant point on the Pareto curve under the latency × accuracy × memory tradeoff this paper measures.
- *V4 (FP4 with software microscaling) is not yet practical* on consumer Blackwell at the granularity we measure. The path to a deployable FP4 attention is the hardware-microscaled `mxf4nvf4` instruction, which requires a CUTLASS-mediated load.
- *The vendor library is not a fixed reference.* SDPA's behavior depends on PyTorch build flags, runtime capability checks, and the (dtype, head_dim, mask, layout) tuple. Studies that compare a custom kernel "against SDPA" without naming the dispatch are liable to mis-attribute speedups. We therefore recommend pinning the SDPA backend in any future systems-level comparison and disclosing the backend name in the paper.

VIII. Limitations

Single hardware platform: Our findings characterize one consumer Blackwell SKU (RTX 5080, sm_120a). The V4 result is most sensitive to the unavailability of `mxf4nvf4` via row-major SMEM; on a setup that links a CUTLASS-built attention kernel through the hardware-microscaled path, we would expect V4 to flip from "slower than V3" to "faster than V3". The recent *SlideSparse* [5] consumer-Blackwell sparse-GEMM evaluation explicitly includes RTX 5080 alongside the 5090, A100, H100, B200, and DGX Spark and demonstrates that the cross- SKU comparisons are tractable; a comparable replication on the attention axis is left as future work. Notably, the recent Knoop-Holtmann consumer-Blackwell LLM-deployment study [6] *omits the RTX 5080 entirely*, covering 5060 Ti, 5070 Ti, and 5090 only—leaving the present paper as the only kernel-level attention study on the SKU at the time of writing.

Single inference scenario: Forward pass only. The backward pass is non-trivial for the narrow-precision variants and out of scope. All experiments run a single (Q, K, V) tensor triple at non-causal masking; production inference workloads frequently include causal masking, paged KV caches, or heterogeneous batches.

Driver and software-version sensitivity: Our results are specific to PyTorch 2.9.1+cu128 on Windows 11 with the driver versions recorded in the artifact's Parquet provenance. The vLLM [19], [20], [22], NVIDIA TransformerEngine [27], and FlashInfer [21] issue trackers document a sustained pattern of attention-dispatch regressions, FP8/NVFP4 precision-sensitivity bugs, and pre-flight-validation false negatives on consumer Blackwell over mid-2026; readers attempting to replicate on a different stack should expect minor numerical and dispatch divergences. We freeze the versions in our artifact and report measured behavior on those exact versions.

Memory ceiling: The 16 GB card means V0 OOMs past seq_len$= 2048$ at our (batch, heads); V1 OOMs past ~ 8192; V2/V3/V4 fit at every measured length. The ceiling forces a fixed $(\text{batch} = 2, \text{heads} = 8)$ rather than a $(\text{batch}, \text{heads})$ sweep.

Precision-format coverage: We test E4M3 (V3) and the FP4 (E2M1) format consumed by `kind::f8f6f4.m16n8k32` (V4). We do *not* characterize E5M2 (the FP8 alternative with one more exponent bit and one fewer mantissa bit), the `mxf4nvf4.m16n8k64` hardware-microscaled NVFP4 path, or the INT4 / INT8 W4A8-style integer paths used in some SageAttention variants. Each is a defensible follow-up.

Clock state: Our consumer-Blackwell host had no admin permission for `nvidia-smi -lgc`; clocks float during the sweep. We report p95/p99 alongside the mean to surface tail variability. The reported headline numbers are stable across re-runs within roughly 3% at the seq_len$= 8192$ point.

IX. Conclusion

We presented a systematic Pareto study of mixed-precision attention on the NVIDIA RTX 5080 (Blackwell, sm_120a), spanning FP32 to NVFP4 across five hand-rolled kernels with a shared algorithmic skeleton. Our headline findings are: an exact 17× memory reduction at the V1→V2 (online softmax + fusion) step; a 2.4× latency reduction at the V2→V3 (FP8 mma.sync) step that closes approximately half of the gap to PyTorch SDPA; and a negative $\approx 30\%$ V3→V4 latency *regression* caused by software- microscaling overhead at the per-mma granularity. The negative V4 result is, to our knowledge, the first quantitative evidence that low-precision attention does

not strictly win on consumer-tier Blackwell, and it points to a concrete next step: lifting V4 onto the hardware-microscaled `mxf4nvf4.m16n8k64` family via a CUTLASS- mediated swizzled-SMEM load.

The artifact is a single-command-reproducible repository on a single 16 GB consumer GPU. Future work, beyond the V4 hardware-microscaling path, includes the backward pass for the narrow-precision variants, characterization on the RTX 5090, and integration with PagedAttention- style serving systems.

REFERENCES

[1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://arxiv.org/abs/2205.14135

[2] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu, "FP8 formats for deep learning," 2022. [Online]. Available: https://arxiv.org/abs/2209.05433

[3] J. Zhang, J. Wei, J. Chen, H. Chen, P. Yu, J. Zhu, and J. Chen, "SageAttention3: Microscaling FP4 attention for inference and an exploration of 8-Bit training," 2025, closest related work to V4 – demonstrates hardware-microscaled NVFP4 attention on RTX 5090 (sm_120a flagship). Cited from arXiv preprint as the venue-published version was not available at submission. [Online]. Available: https://arxiv.org/abs/2505.11594

[4] P. Zhang, M. Noto, W. Tan, C. Jiang, W. Lin, W. Zhou, and H. Zhang, "Attn-QAT: 4-bit attention with quantization-aware training," *arXiv preprint arXiv:2603.00040*, 2026, first systematic study of 4-bit QAT for attention; demonstrates that the natural FP4 forward + high-precision Flash Attention backward recipe is unstable; identifies (1) matching low-precision recomputation in backward and (2) resolving FA's implicit precision assumptions as the two principles for stable FP4 attention. [Online]. Available: https://arxiv.org/abs/2603.00040

[5] H. Shao, Y. Hao, T. Song, Y. Xia, D. Zhang, S. Huang, X. Wu, S. Xu, L. Xu, L. Dong, Z. Chi, Y. Zou, and F. Wei, "SlideSparse: Fast and flexible (2N-2):2N structured sparsity," *arXiv preprint arXiv:2603.05232*, 2026, recent (March 2026) consumer-Blackwell systems paper; explicitly evaluates on RTX 5080 alongside A100, H100, B200, RTX 4090, and DGX Spark. Targets sparse GEMM, not attention. [Online]. Available: https://arxiv.org/abs/2603.05232

[6] J. Knoop and H. Holtmann, "Private LLM inference on consumer blackwell GPUs: A practical guide for cost-effective local deployment in SMEs," *arXiv preprint arXiv:2601.09527*, 2026, systematic evaluation of NVIDIA Blackwell consumer GPUs — but only RTX 5060 Ti, 5070 Ti, and 5090. The RTX 5080 is explicitly omitted, leaving the present paper as the only kernel-level attention study targeting that SKU. [Online]. Available: https://arxiv.org/abs/2601.09527

[7] H. Qiu and Q. Yao, "Why low-precision transformer training fails: An analysis on flash attention," *arXiv preprint arXiv:2510.04212*, 2025, mechanistic explanation of catastrophic loss explosion in low-precision flash attention training; identifies the joint effect of (a) emergence of similar low-rank representations in attention and (b) compounding biased rounding errors. ICLR 2026 poster. [Online]. Available: https://arxiv.org/abs/2510.04212

[8] Y. Zhang, Z. Su, S. Hu, R. Yang, W. Wu, Y. Qian, Y. Xie, and X. Cai, "SnapMLA: Efficient long-context MLA decoding via hardware-aware FP8 quantized pipelining," *arXiv preprint arXiv:2602.10718*, 2026, identifies FP8 attention sub-types in DeepSeek MLA decoding (RoPE-bearing components, KV-shared structure) requiring per-component precision retention. Authored by Meituan LongCat Team + Tsinghua University. [Online]. Available: https://arxiv.org/abs/2602.10718

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[10] NVIDIA Corporation, "Parallel thread execution ISA version 8.7," Technical specification, 2024. [Online]. Available: https://docs.nvidia.com/cuda/parallel-thread-execution/

[11] NVIDIA CUTLASS Team, "CUTLASS: CUDA templates for linear algebra subroutines, v4.4.2," https://github.com/NVIDIA/cutlass, 2024.

[12] PyTorch Team, "`torch.nn.functional.scaled_dot_product_attention` documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html, 2024, documents the four backend dispatch paths (FLASH_ATTENTION, CUDNN_ATTENTION, EFFICIENT_ATTENTION, MATH) and the `sdpa_kernel` pinning context manager.

[13] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: https://arxiv.org/abs/2407.08608

[14] NVIDIA Corporation, "NVIDIA RTX Blackwell GPU Architecture Whitepaper," Technical whitepaper, 2025. [Online]. Available: https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf

[15] J. Dong, B. Feng, D. Guessous, Y. Liang, and H. He, "Flex Attention: A programming model for generating optimized attention kernels," in *Proceedings of MLSys*, 2025, compiler-driven programming model that compiles user-defined attention masks via torch.compile; alternative path to handwritten kernels. [Online]. Available: https://arxiv.org/abs/2412.05496

[16] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[17] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, "vAttention: Dynamic memory management for serving LLMs without PagedAttention," *arXiv preprint arXiv:2405.04437*, 2024, documents PagedAttention kernel decode-time regressions vs. FlashAttention-2 baseline. ASPLOS '25. [Online]. Available: https://arxiv.org/abs/2405.04437

[18] J. Zhang, J. Wei, H. Huang, P. Zhang, J. Zhu, and J. Chen, "SageAttention: Accurate 8-bit attention for plug-and-play inference acceleration," in *International Conference on Learning Representations (ICLR)*, 2025, establishes FP8 attention practicality; outperforms FlashAttention-2 by 2.1x and xformers by 2.7x on Hopper. Tsinghua University. [Online]. Available: https://arxiv.org/abs/2410.02367

[19] vLLM Project, "Issue #14452: Steps to run vLLM on RTX 5080 or 5090," https://github.com/vllm-project/vllm/issues/14452, 2025, documents the CUDA 12.8 + container build requirements for working consumer Blackwell vLLM deployment, and that FA3 backend does not work with Blackwell yet.

[20] ——, "Issue #29030: vLLM 0.11.1 undefined symbol `cutlass_moe_mm_sm100` on RTX 5080 (SM 12.0) with CUDA 13.0," https://github.com/vllm-project/vllm/issues/29030, 2025, documents that SM 12.0 (consumer Blackwell) is not a superset of SM 10.0 (datacenter Blackwell): SM10 kernels do not run on SM12 hardware.

[21] ——, "Issue #35138: Qwen3.5-397B-A17B-FP8 has accuracy issues with FlashInfer attention backend on blackwell," https://github.com/vllm-project/vllm/issues/35138, 2026, documents Blackwell-specific FP8 attention correctness divergence in FlashInfer; recommended workaround is backend switch to Flash-Attn rather than precision change.

[22] ——, "Issue #30707: RTX 5080 (SM120) + NVFP4 model fails pre-flight memory check despite fitting in VRAM," https://github.com/vllm-project/vllm/issues/30707, 2025, documents over-aggressive 16 GB consumer Blackwell pre-flight validation rejecting NVFP4-quantized models that fit in VRAM.

[23] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021, argues code release alone is insufficient; introduces the reproducibility checklist later adopted by major ML conferences. [Online]. Available: https://arxiv.org/abs/2003.12206

[24] A. Desai, M. Abdelhamid, and N. R. Padalkar, "What is reproducibility in artificial intelligence and machine learning research?" *AI Magazine*, 2025, formalizes the distinctions among repeatability, dependent / independent reproducibility, and direct vs. conceptual replication. Our paper is a conceptual replication under hardware shift. [Online]. Available: https://arxiv.org/abs/2407.10239

[25] T. Kalibera and R. E. Jones, "Rigorous benchmarking in reasonable time," in *Proceedings of the 2013 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, 2013, pp. 63–74, steady-state detection,

multi-source-of-variance accounting, and required-iteration computation for systems benchmarking.
[26] E. van der Kouwe, D. Andriesse, H. Bos, C. Giuffrida, and G. Heiser, "Benchmarking crimes: An emerging threat in systems security," *arXiv preprint arXiv:1801.02381*, 2018, catalogs 22 specific methodological errors in systems benchmarking, surveyed across 50 defense papers in top venues. Used as defensive checklist in our experimental setup. [Online]. Available: https://arxiv.org/abs/1801.02381
[27] NVIDIA Transformer Engine Project, "Issue #2186: Fused attention with THD format + CP may cause NaN in backward; test-suite documents FP8 attention skip list," https://github.com/NVIDIA/TransformerEngine/issues/2186, 2025, documents both a backward-pass NaN bug and the explicit FP8-attention skip list (THD layout, bias, sliding window, MLA-CP) in the official test suite.

APPENDIX A
REPRODUCIBILITY

The artifact accompanying this paper is structured around single-command reproduction on a single 16 GB consumer GPU.

A. Software Environment

- NVIDIA Driver: as recorded in `bench/results/sweep_final.parquet` (`driver_version` column) and per-figure provenance.
- CUDA Toolkit: 12.8.
- PyTorch: 2.9.1+cu128.
- Visual Studio Build Tools 2022 (MSVC, Desktop development with C++).
- Ninja, in the conda `gpu` environment.

B. Build

From the repo root, with the project's PowerShell helper sourced (`. ./env/activate_for_build.ps1`):

```
pip install -r env/requirements.txt
pip install --no-build-isolation -e kernels/
    v0_naive_fp32
pip install --no-build-isolation -e kernels/
    v1_tiled_fp16
pip install --no-build-isolation -e kernels/
    v2_flash_fp16
pip install --no-build-isolation -e kernels/
    v3_flash_fp8
pip install --no-build-isolation -e kernels/
    v4_flash_nvfp4
```

V4's setup targets `sm_120a`; the others target plain `sm_120`.

C. Sweep

```
python bench/run_all.py \
  --config bench/configs/sweep_final.yaml \
  --output bench/results/sweep_final.parquet
```

The output Parquet has one row per timed iteration with full provenance (driver, CUDA, torch, GPU, timestamp, git commit). The sweep takes approximately 30–45 minutes on RTX 5080.

D. Figures

```
python analysis/paper_figures.py
```

Regenerates every figure in this paper from `bench/results/sweep_final.parquet`, written to `paper/figures/` as PDF (vector) and PNG (preview).

E. Tests

```
pytest
```

Runs the full correctness test suite (∼495 tests across the five variants plus the harness). Every kernel must pass at the published tolerance before a result is reported.

F. Known Failure Modes

- "DLL load failed while importing _C" on import: `import torch` *before* importing any compiled kernel.
- `ERR_NVGPUCTRPERM` from Nsight Compute: needs admin on consumer GPUs. Use `cuobjdump -dump-sass` for Tensor-Core engagement instead.
- PowerShell wraps native-command stderr as `NativeCommandError` with exit code 1 even on successful runs. Do not chase. Run from `cmd.exe` or accept the noise.
- Editable install prints a tiny wheel size (∼3 KB); the actual `.pyd` (∼200–540 KB) lands in the kernel directory itself.

G. Artifact Contents

- `kernels/v{0..4}_*/` — the five attention kernels, each with its own `setup.py` and at least one passing correctness test in `tests/`.
- `bench/` — the harness, run drivers, and sweep YAML configs.
- `analysis/` — figure-generation scripts.
- `paper/` — this manuscript and its figures.
- `external/cutlass/` — CUTLASS submodule pinned at v4.4.2 (reference-only; not linked at build time).